

The 11-Day Impossibility: Engineering at AI Scale

A forensic case study of lib-pcb: How one developer executed a production-grade 18-month project in under two weeks.

```
Project: lib-pcb  
Version: 1.0.0-RELEASE  
Status: PRODUCTION READY  
Author: Thor Henning Hetland  
Build Date: January 2026  
Duration: 11 Days
```


44x to 200x Velocity Increase Over Industry Standard

INDUSTRY STANDARD

- Timeline:
9–24 Months
- Team:
3–5 Developers
- Est. Cost:
\$225,000 – \$450,000
- Output:
Standard Release Cycle

KEY METRIC:
44–200x
VELOCITY INCREASE

THE EVENT (lib-pcb)

- Timeline:
11 Days
- Team:
1 Developer
- Actual Cost:
\$6,350 (Compute & API)
- Output:
55 Production Releases

Context: Production-grade Java library for PCB manufacturing (197k LOC) with zero corners cut.

This Was Not a 'Toy App'. It Was Rigorous Production Engineering.

Dashboard

SCALE

Total Lines: 197,831

Java Files: 683

Production Code:
80,644 LOC

RIGOR

Test Code: 117,187 LOC

Test Ratio:
1.45:1 (Test-to-Code)

Total Tests: 7,461

QUALITY

Pass Rate: 99.8%

Production Bugs: 0

CI/CD Status: **PASSING**

Repository Public. Commit history
verifies 474 commits from Jan 16-26.
github.com/exoreaction/lib-pcb

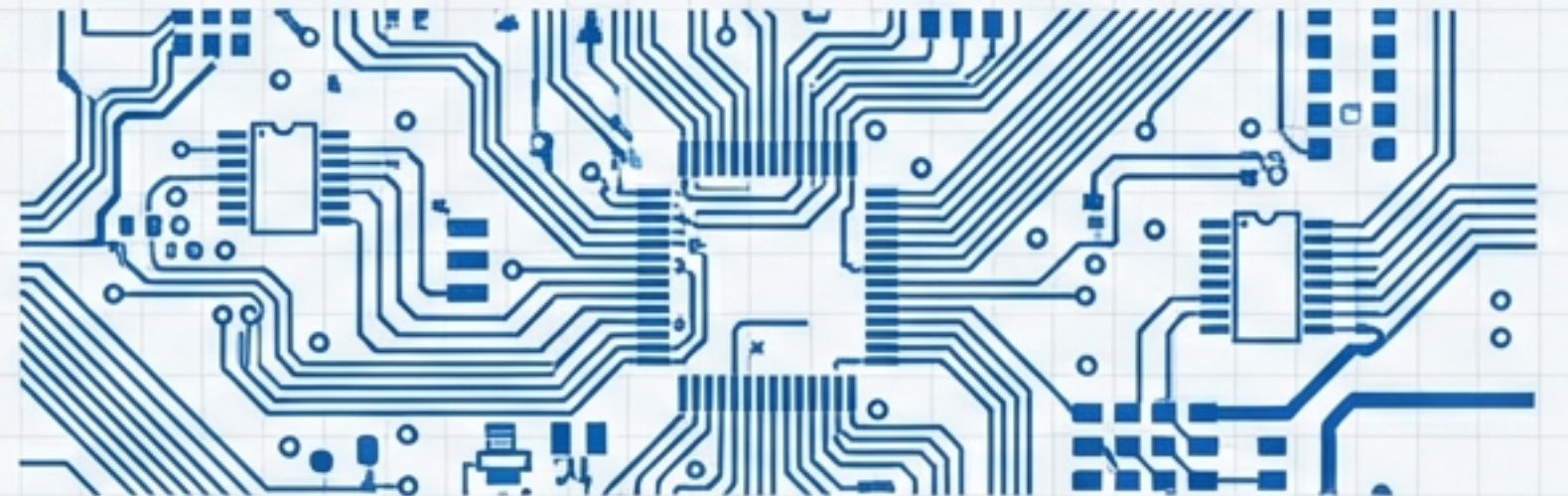
'Domain Complexity vs. Standard AI Demos' in Inter Tight

STANDARD AI DEMO

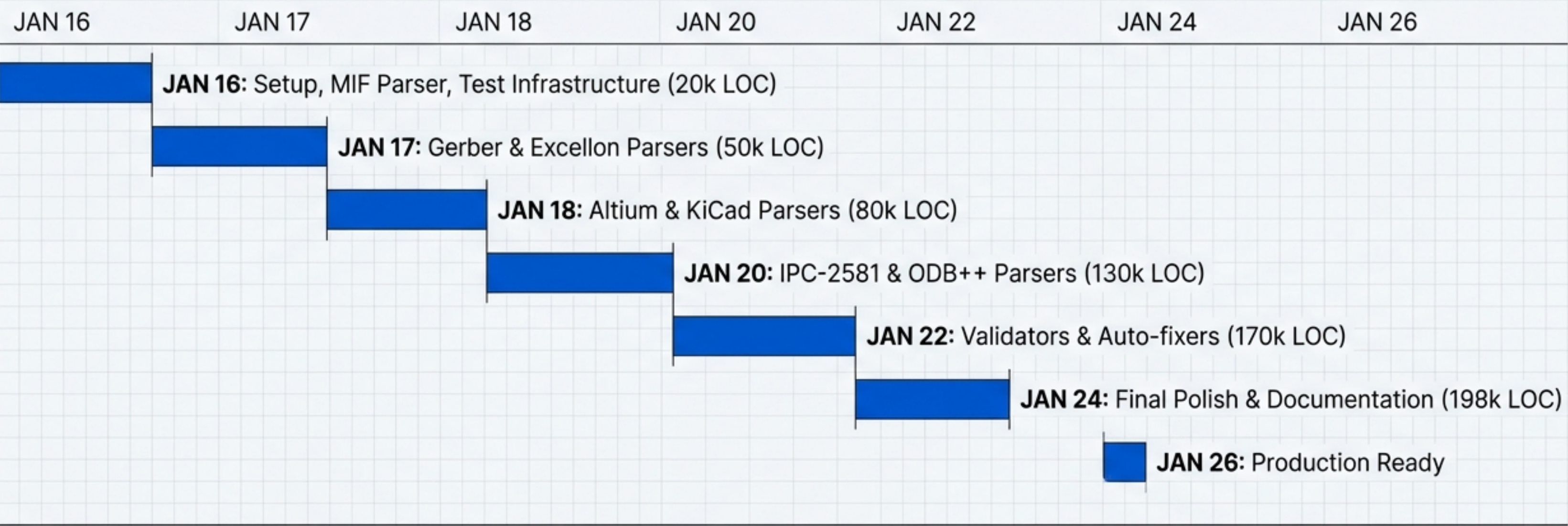
- Generic CRUD APIs
- Simple React Frontends
- Todo Apps
- Verdict: Low Risk, Low Precision.

LIB-PCB REALITY

- **Byte-Level Binary Parsing:** One bit wrong corrupts manufacturing.
- **8 Industry Parsers:** MIF, Gerber, Excellon, Altium, IPC-2581, KiCad, ODB++.
- **Regulatory Compliance:** 28 Validator classes enforcing IPC-6012 standards.
- **Interoperability:** Hub-and-spoke architecture converting proprietary formats.



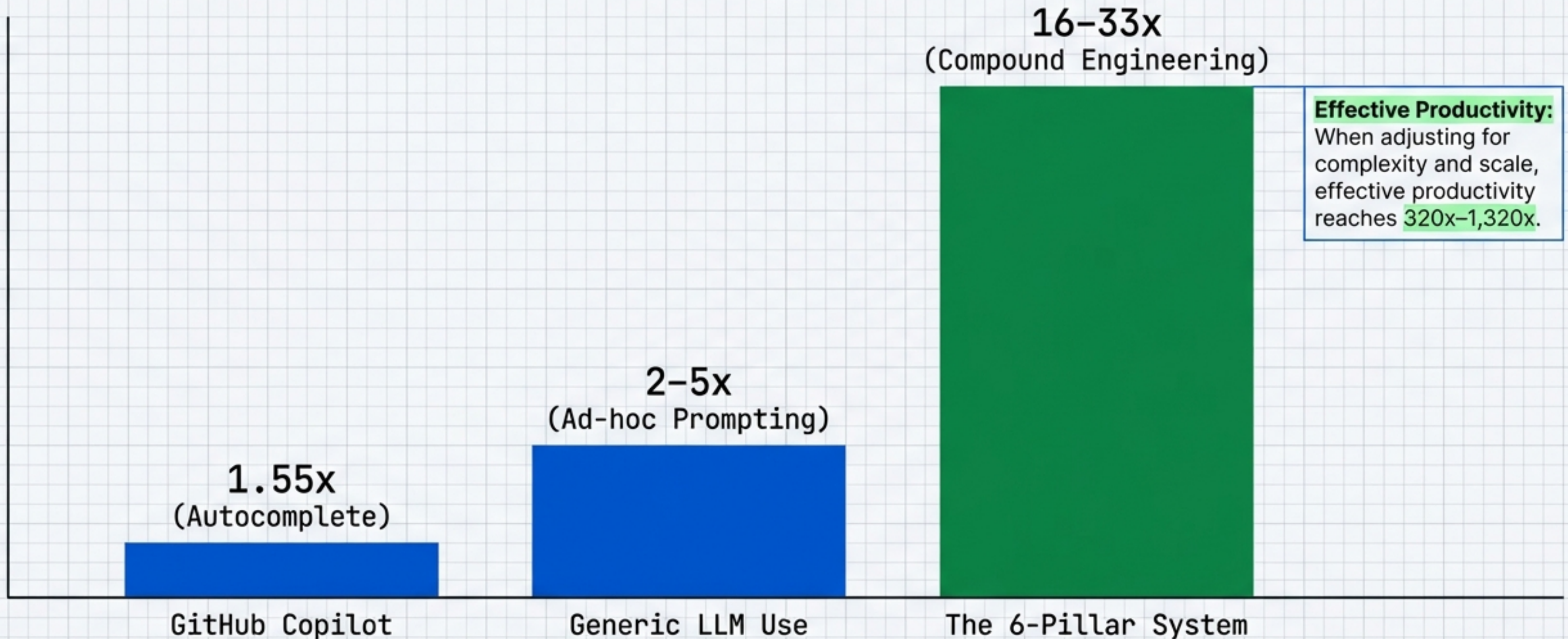
11 Days of Compressed Velocity



INDUSTRY STANDARD

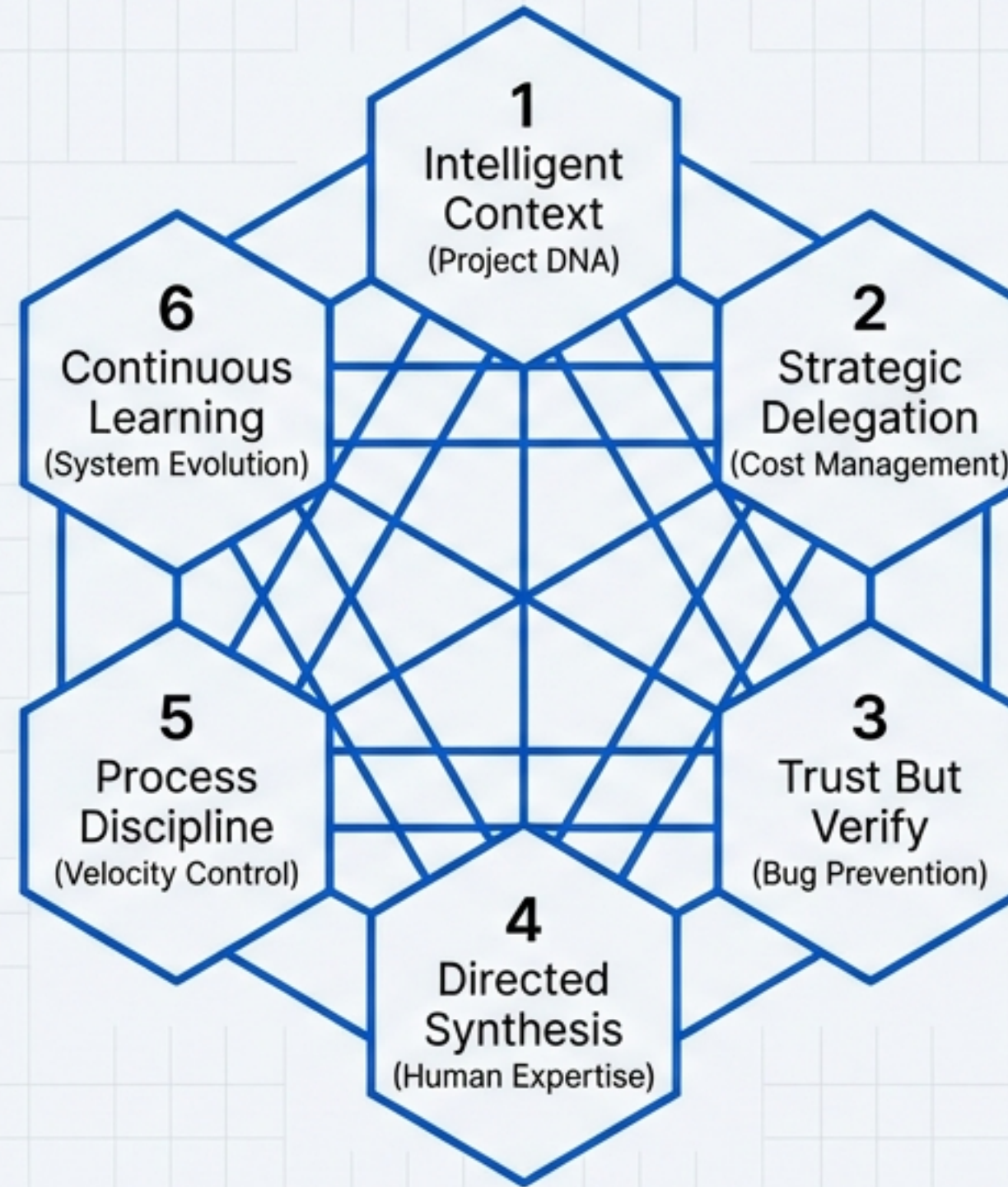
JetBrains Mono
Traditional industry timeline typically requires 4-6 months just for the first parser.

The “Secret Sauce” Is Not the Tool. It Is the System.



Giving a novice a hammer doesn't build a house. The difference is the methodology.

The System for AI-Scale Engineering



These pillars multiply, they don't add. Missing one collapses the compound effect.

Teaching the AI & Managing the Budget

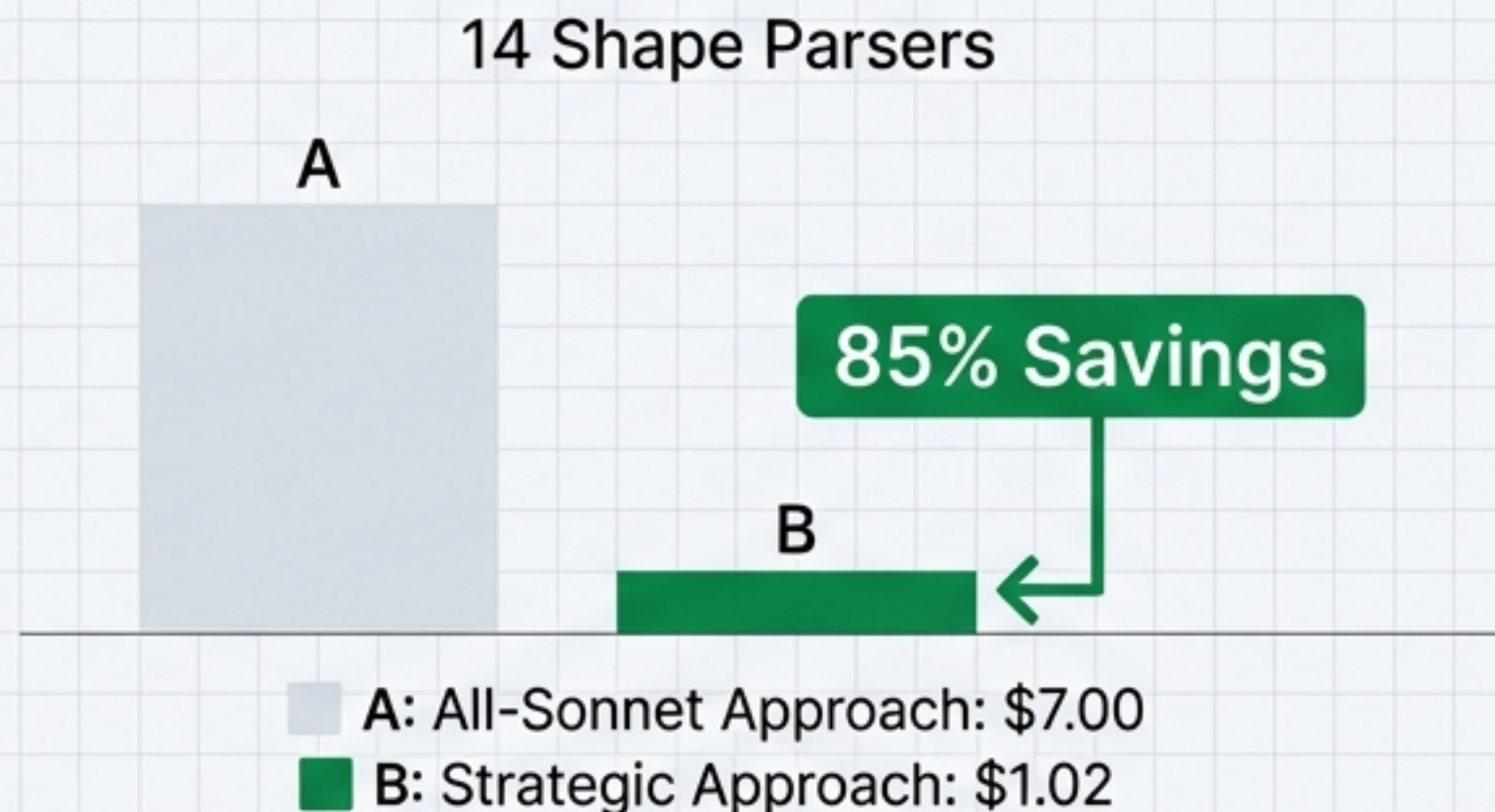
Pillar 1: Intelligent Context



767 lines of
"Project DNA"

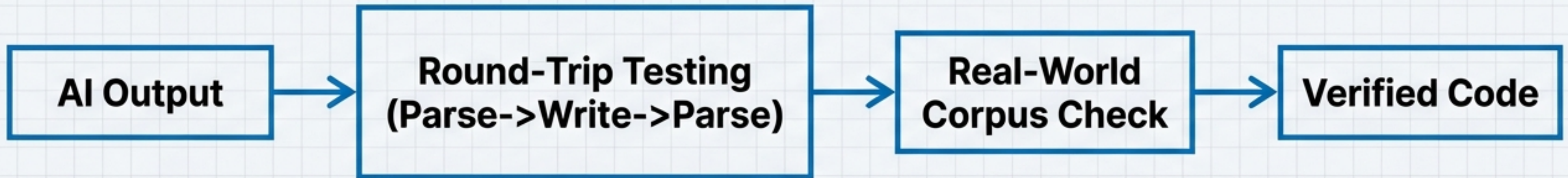
- Contains architectural decisions, anti-patterns, and 54 domain-specific skill files (e.g., "How to parse binary headers" in JetBrains Mono).
- Generic AI knows Java; it doesn't know YOUR architecture.

Pillar 2: Strategic Delegation



Use smart model (Sonnet in JetBrains Mono) to design the pattern -> Use cheap model (Haiku in JetBrains Mono) to replicate it 13 times.

Trust But Verify: How to Catch 100% of Hallucinations



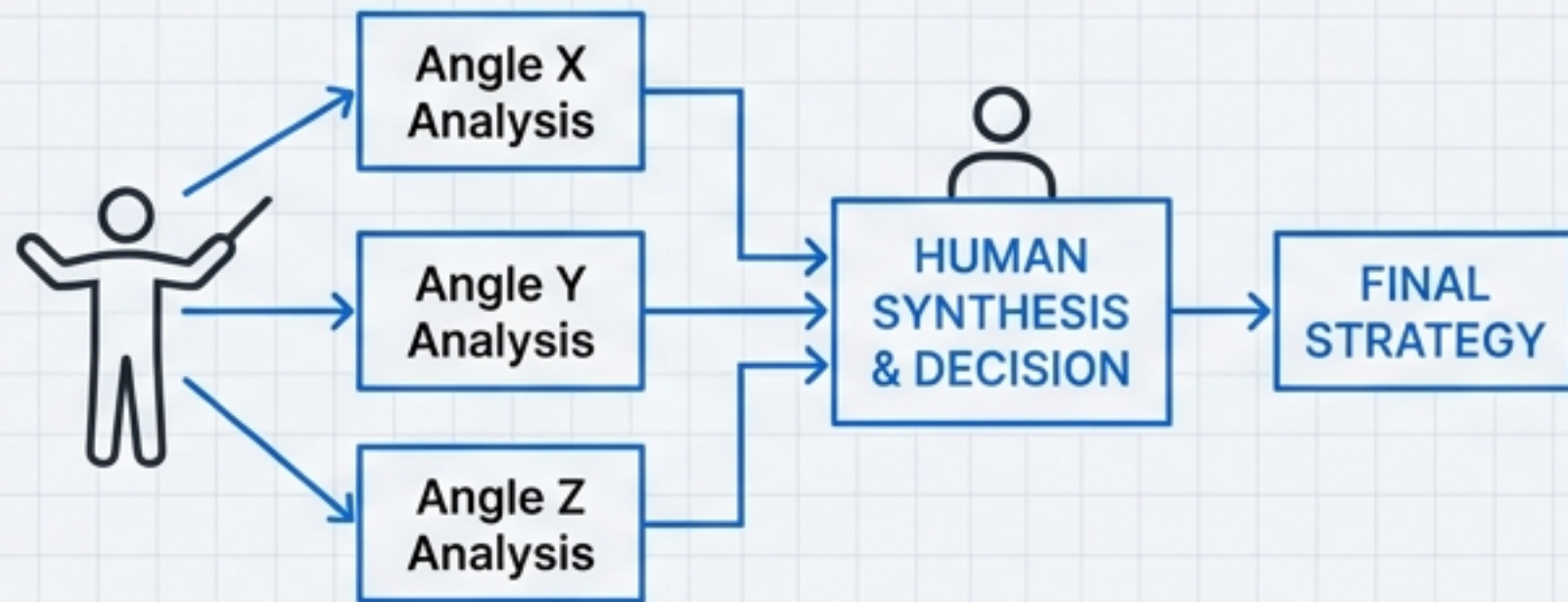
1. **Round-Trip Testing:** Catches data corruption instantly.
2. **Real-World Corpus:** Testing against 191 actual PCB files.
3. **Property-Based Testing:** Checking invariants (e.g., “Trace width never negative”).
4. **Differential Testing:** Compare outputs against KiCad/Altium.
5. **AI Reviewing AI:** Cheap models reviewing code for \$0.07.

Result: 15+ critical bugs caught in seconds pre-merge. 0 bugs in production.

Directed Synthesis & Process Discipline

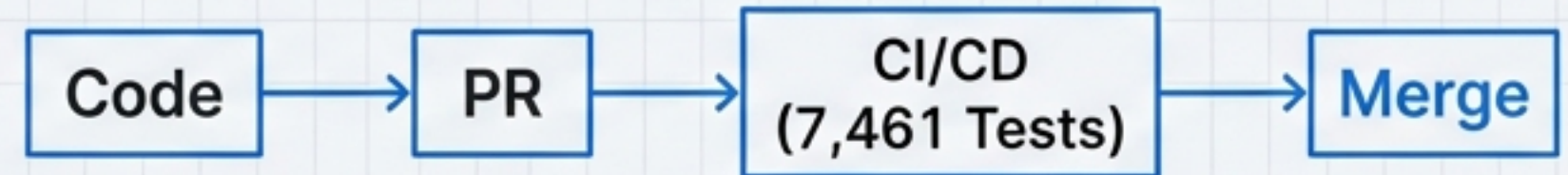
Pillar 4: Directed Synthesis

Concept: You are the conductor, AI is the orchestra. Do not ask "What should I do?". Ask "Analyze from Angle X and Angle Y", then YOU synthesize.



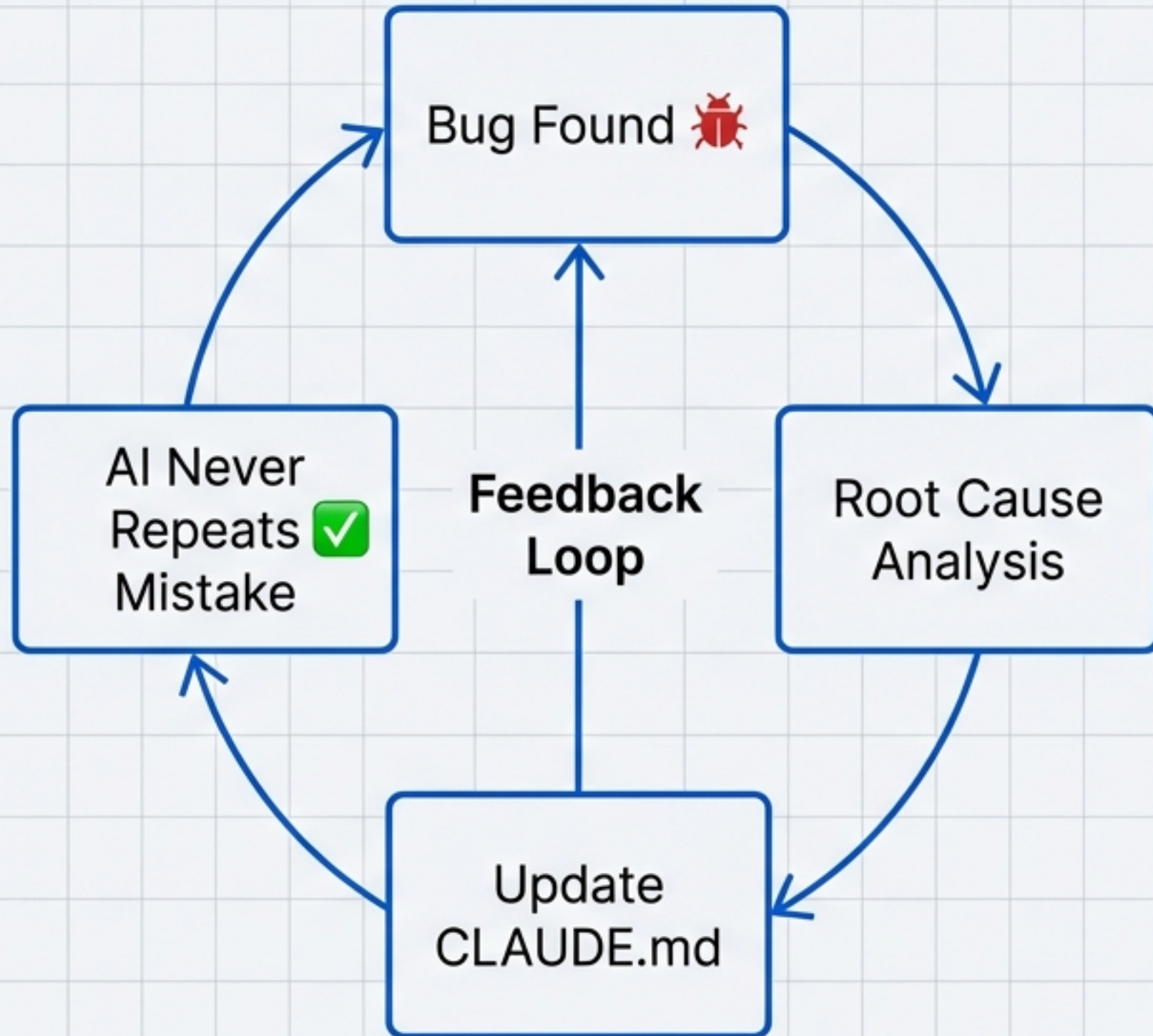
Pillar 5: Process Discipline

Rule: 100% PR workflow, automated CI/CD, 0 direct commits to main.



Paradox: Stricter process = faster delivery (**150+** PRs with **<4 hour** cycle time).

Continuous Learning: The Compound Effect



Evolution of Context

Day 1	200-line Context file	5 Skills
Month 3	1,200-line Context file	51 Skills

Key Insight:

The AI gets smarter every single day of the project. An 8x productivity improvement from Month 1 to Month 3 driven purely by system learning.

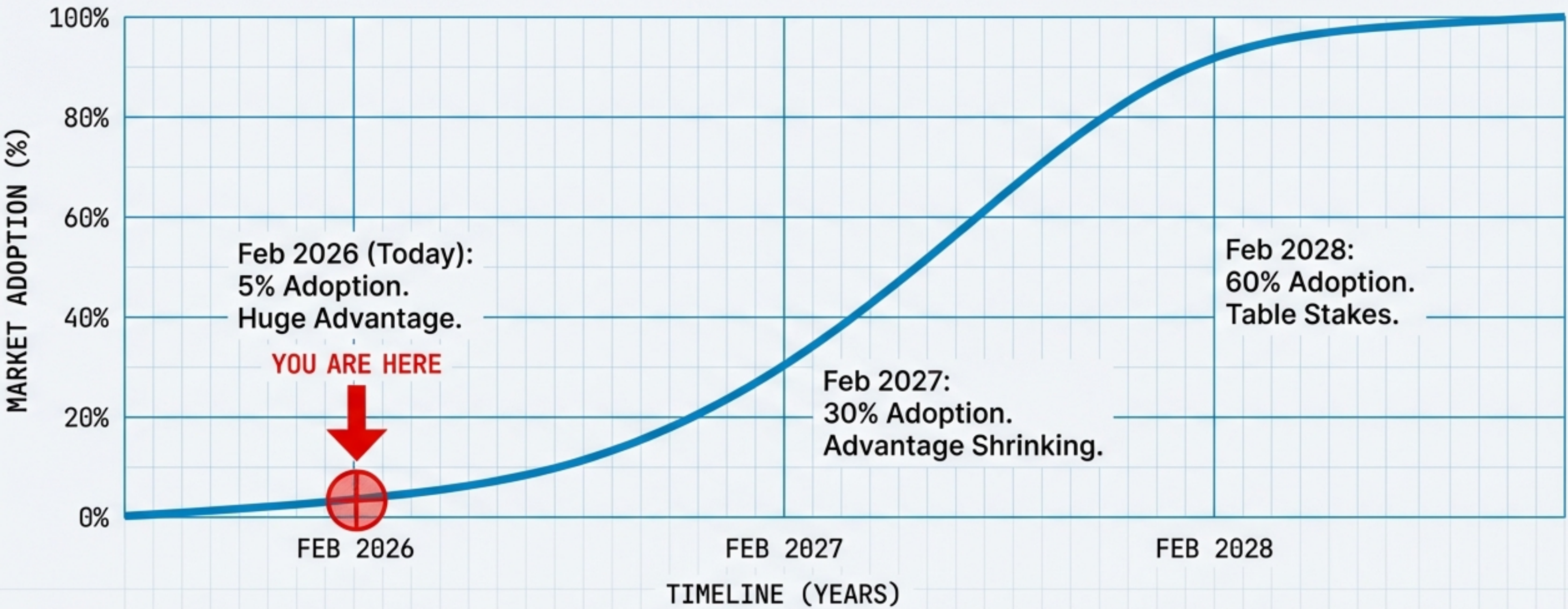
The Economics of 8x Velocity

Traditional Method: 6 Weeks				
Design	Dev	QA	Fix	Deploy

New Method: 4-5 Days		
Design	AI Implementation	Auto-Verify

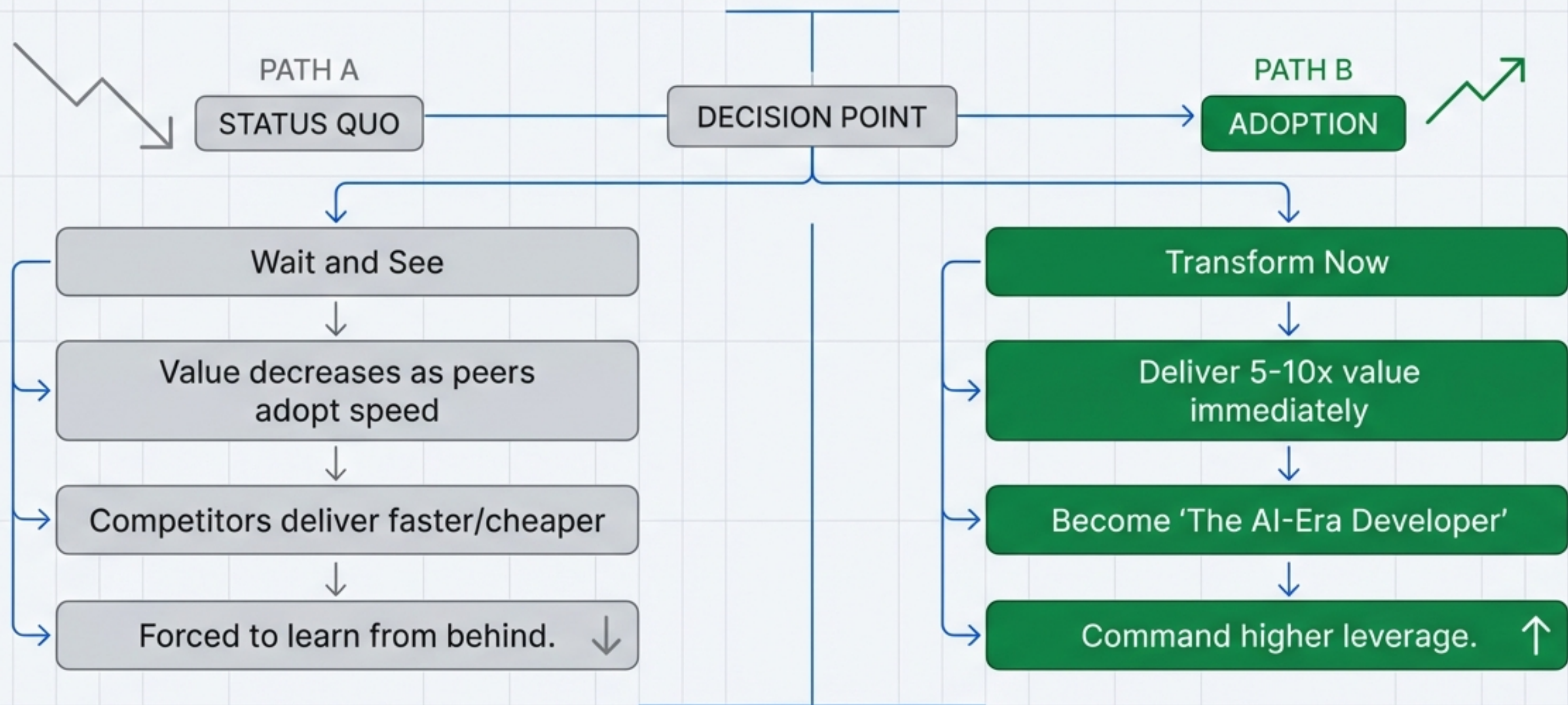
- Higher margins for consultancy.
- Ability to clear backlog and tackle **8-12 projects/year** vs **1-2**.
- Elimination of the “Fast vs. Good” trade-off.

Your Competitive Window is Closing



Warning: You have 12–18 months to lead before this is the standard baseline.

Adapt or Obsolesce: Two Paths Forward



"The gap widens every month."

JetBrains Mono

The Wake-Up Call is Over. The Work Begins.

Three Implementation Paths



Offer: 2-Day Hands-On Workshop + 3-Month Mentoring on YOUR actual codebase.

Bottom Action: Audit the code yourself: github.com/exoreaction/lib-pcb

Contact Footer Thor Henning Hetland | thor@exoreaction.com